Code

```
pragma solidity ^0.8.12;

   function addProduct(bytes32 _manufactuerID, bytes32 _productName, bytes32
_productSN, bytes32 _productBrand,

   uint256 _productPrice) public{

      productItems[productCount] = productItem(productCount, _productSN, _productName,
_productBrand,

      _productPrice, "Available");

      productMap[_productSN] = productCount;

      productCount++;

      productsManufactured[_productSN] = _manufactuerID;

   }

   function viewProductItems () public view returns(uint256[] memory, bytes32[] memory,
bytes32[] memory, bytes32[] memory, uint256[] memory, bytes32[] memory) {

      uint256[] memory pids = new uint256[](productCount);

      bytes32[] memory pSNs = new bytes32[](productCount);

      bytes32[] memory pnames = new bytes32[](productCount);

      bytes32[] memory pbrands = new bytes32[](productCount);

      uint256[] memory pprices = new uint256[](productCount);

      bytes32[] memory pstatus = new bytes32[](productCount);


      for(uint i=0; i<productCount; i++){

         pids[i] = productItems[i].productId;

         pSNs[i] = productItems[i].productSN;

           pnames[i] = productItems[i].productName;

         pbrands[i] = productItems[i].productBrand;

         pprices[i] = productItems[i].productPrice;

         pstatus[i] = productItems[i].productStatus;

      }

      return(pids, pSNs, pnames, pbrands, pprices, pstatus);
```

```
    }
    function manufacturerSellProduct(bytes32 _productSN, bytes32 _sellerCode) public{
        productsWithSeller[_sellerCode].push(_productSN);
        productsForSale[_productSN] = _sellerCode;
    }
    function sellerSellProduct(bytes32 _productSN, bytes32 _consumerCode) public{
        bytes32 pStatus;
        uint256 i;
        uint256 j=0;
        if(productCount>0) {
            for(i=0;i<productCount;i++) {
                if(productItems[i].productSN == _productSN) {
                    j=i;
                }
            }
        }
         pStatus = productItems[j].productStatus;
        if(pStatus == "Available") {
            productItems[j].productStatus = "NA";
            productsWithConsumer[_consumerCode].push(_productSN);
            productsSold[_productSN] = _consumerCode;
        }
    }
    function queryProductsList(bytes32 _sellerCode) public view returns(uint256[] memory,
bytes32[] memory, bytes32[] memory, bytes32[] memory, uint256[] memory, bytes32[]
memory){
        bytes32[] memory productSNs = productsWithSeller[_sellerCode];
        uint256 k=0;
        uint256[] memory pids = new uint256[](productCount);
        bytes32[] memory pSNs = new bytes32[](productCount);
```

```
        bytes32[] memory pnames = new bytes32[](productCount);
        bytes32[] memory pbrands = new bytes32[](productCount);
        uint256[] memory pprices = new uint256[](productCount);
        bytes32[] memory pstatus = new bytes32[](productCount);

        for(uint i=0; i<productCount; i++){
            for(uint j=0; j<productSNs.length; j++){
                if(productItems[i].productSN==productSNs[j]){
                    pids[k] = productItems[i].productId;
                    pSNs[k] = productItems[i].productSN;
                    pnames[k] = productItems[i].productName;
                    pbrands[k] = productItems[i].productBrand;
                    pprices[k] = productItems[i].productPrice;
                    pstatus[k] = productItems[i].productStatus;
                    k++;
                }
            }
        }
        return(pids, pSNs, pnames, pbrands, pprices, pstatus);
    }
    function querySellersList (bytes32 _manufacturerCode) public view returns(uint256[] memory, bytes32[] memory, bytes32[] memory, bytes32[] memory, uint256[] memory, bytes32[] memory, bytes32[] memory) {
        bytes32[] memory sellerCodes = sellersWithManufacturer[_manufacturerCode];
        uint256 k=0;
        uint256[] memory ids = new uint256[](sellerCount);
        bytes32[] memory snames = new bytes32[](sellerCount);
        bytes32[] memory sbrands = new bytes32[](sellerCount);
        bytes32[] memory scodes = new bytes32[](sellerCount);
        uint256[] memory snums = new uint256[](sellerCount);
```

```
    bytes32[] memory smanagers = new bytes32[](sellerCount);
    bytes32[] memory saddress = new bytes32[](sellerCount);

    for(uint i=0; i<sellerCount; i++){
      for(uint j=0; j<sellerCodes.length; j++){
        if(sellers[i].sellerCode==sellerCodes[j]){
          ids[k] = sellers[i].sellerId;
          snames[k] = sellers[i].sellerName;
          sbrands[k] = sellers[i].sellerBrand;
          scodes[k] = sellers[i].sellerCode;
          snums[k] = sellers[i].sellerNum;
          smanagers[k] = sellers[i].sellerManager;
          saddress[k] = sellers[i].sellerAddress;
          k++;
          break;
        }
      }
    }
    return(ids, snames, sbrands, scodes, snums, smanagers, saddress);
  }
  function getPurchaseHistory(bytes32 _consumerCode) public view returns (bytes32[]
memory, bytes32[] memory, bytes32[] memory){
    bytes32[] memory productSNs = productsWithConsumer[_consumerCode];
    bytes32[] memory sellerCodes = new bytes32[](productSNs.length);
    bytes32[] memory manufacturerCodes = new bytes32[](productSNs.length);
    for(uint i=0; i<productSNs.length; i++){
      sellerCodes[i] = productsForSale[productSNs[i]];
      manufacturerCodes[i] = productsManufactured[productSNs[i]];
    }
    return (productSNs, sellerCodes, manufacturerCodes);
```

```
    }
    //Verify
   function verifyProduct(bytes32 _productSN, bytes32 _consumerCode) public view
returns(bool){
       if(productsSold[_productSN] == _consumerCode){
           return true;
        }
        else{
           return false;
        }
    }
}
```